

Recruitment Task — Extending the Platform with Analytics and Reporting Modules

Context

You are given access to the **candidate-workspace** repository — a working user management platform called **UserHub** (pnpm monorepo: NestJS + Prisma + PostgreSQL in `apps/api`, Next.js App Router + Tailwind in `apps/web`, a mock external text-analysis API in `apps/mock-analytics`).

The system core (JWT authentication, `ADMIN/MANAGER/ANALYST` roles, user CRUD, Core API, frontend foundation) is implemented and tested. Treat it as code written by the team you are joining — your job is to **extend it, not rebuild it**.

The Task

Implement the two modules described below. Edge cases are the heart of this task — they are what separates an engineering solution from a plain CRUD.

Module 1: AI Analytics (`analytics`)

The module integrates with an **external text-analysis API** (in the dev environment: `apps/mock-analytics`, with failure modes described in its README) that returns sentiment and keywords for a given text.

Functional requirements:

- `POST /api/v1/analytics/analyze` — accepts a text (e.g. a note about a user), sends it to the external API, and persists the analysis result in the database, linked to the user.
- `GET /api/v1/analytics/users/:id` — analysis history for a given user (paginated).
- View in `apps/web`: an analysis form + a list of results with clear loading and error states (native Next.js files).

Required edge cases:

1. **The external API does not respond** (timeout) or returns 5xx — the application **MUST NOT** return a raw 500. Required: request timeout, **retry with backoff (max 2 retries)**, and once retries are exhausted — a clear domain error (`ANALYTICS_PROVIDER_UNAVAILABLE`, HTTP 503) via the global filter. The user's request and the fact of failure must be persisted (status `FAILED`) so it can be retried.

2. **The external API returns a partial or schema-invalid response** (e.g. missing the `sentiment` field) — the response MUST be validated on the backend; incomplete data must never land in the database as a "successful" analysis.
3. **Idempotency**: re-submitting identical text for the same user within a short time window must not create a duplicate analysis — the mechanism (content hash? time window?) is a design decision: design it and document the rationale in the module README.
4. The frontend must distinguish between states: "analysis in progress", "failed with retry available", "result ready" — without manual `isLoading` flags wherever App Router mechanisms suffice.

Module 2: Reporting (`reports`)

The module generates aggregate reports from user and analysis data.

Functional requirements:

- `POST /api/v1/reports` — request generation of a report for a date range (computed asynchronously or synchronously — a design decision to be documented).
- `GET /api/v1/reports/:id` — fetch a report (metadata + aggregated data: analysis count, sentiment distribution, most active users).
- Report list view + report details view in `apps/web`.

Required edge cases:

1. **Contextual authorization**: access to reports depends on a combination of conditions, not on the role alone:
 - `ADMIN` — sees all reports;
 - `MANAGER` — sees reports they requested themselves; NEVER sees reports covering ONLY inactive users (archival data reserved for ADMIN) — even if they requested them themselves;
 - `ANALYST` — may request reports, but only sees aggregated data in them (no list of user names/e-mails) — same endpoint, different response shape depending on the role;
 - a user with `isActive: false` — no access whatsoever (even with a valid, non-expired JWT). A plain `RolesGuard` is not enough — an authorization layer aware of the resource context is required, with unit tests for every combination.
2. **Date range with no data** — an "empty" report is a valid result (200, not 404), explicitly flagged with `isEmpty`.
3. **Invalid ranges** — `dateFrom > dateTo`, a range in the future, a range longer than 365 days → validation at the DTO level with clear messages (no logic in the controller).
4. **A report referencing FAILED analyses** — decide and document whether they count towards aggregations; the decision must be consistent and covered by a test.

Working Rules

- **Team standards are non-negotiable.** The repository contains context files for AI and humans: `.github/copilot-instructions.md`, `.github/instructions/*.instructions.md` (NestJS, Next.js, workflow standards), `AGENTS.md`. Code that violates them is treated like code that fails code review.
- **Using AI (Copilot, Claude, Cursor, etc.) is allowed and expected.** Among other things, we evaluate how effectively you make the AI follow the team standards and how you verify its output — not the mere fact of using it.
- **Leave the context files you use when working with AI in the repository** — including your own `goals.md` (or equivalent) translating the task requirements into a source of truth for the AI, plus any additional instructions or prompts. This is part of the evaluation: we want to see HOW you work with AI, not just the end result. The files in the repo assume such a requirements document exists — creating and maintaining it is up to you.
- **Tests are a hard requirement** (details in `workflow.instructions.md` §3): unit tests for business logic, endpoint tests including error cases, tests for components with conditional logic. A feature without tests is unfinished.
- **Commit history matters:** small, logical commits following the `type(scope) : description` convention. One giant "add modules" commit disqualifies this evaluation area.
- **Document design decisions** in the module README — wherever the task leaves a choice (idempotency mechanism, sync/async reports, treatment of `FAILED` analyses in aggregations), we evaluate the rationale, not the specific choice. Ambiguities in the requirements are partially intentional — document your assumptions instead of guessing.

Priorities When Time Runs Out

We evaluate **depth, not breadth**. It is better to deliver the `analytics` module in full (with edge cases and tests) and implement `reports` partially — in that case, describe the missing scope as a design write-up (decisions, authorization model, test plan) in the module README. Two half-baked modules is the worst possible outcome.

How We Evaluate

Area	What we check
Runnability	The project starts from a fresh clone using the README commands (including migrations and seed); tests pass with a single command — checked first

Area	What we check
Standards compliance	Whether the code follows .github/instructions/ (entity modules, strict DTOs, exception filter, Server/Client Components) or looks tutorial-grade
Edge cases	Whether ALL edge cases from this document are handled and tested
Tests	Coverage of business logic and endpoints, including failure paths; quality of assertions, not the raw number of tests
Work history	Small, logical commits — a readable history instead of one giant commit
Working with AI	Context files left in the repo (e.g. goals.md), documented design rationale, traces of verifying AI output

Logistics

1. Clone the repository:
<https://github.com/FOTC-org/candidate-workspace>.
2. Push your code to **your own private repository** on GitHub (not a fork — forks of a public repo are public).
3. Add [kczajka](#) as a collaborator (read) when you are done.

Running the project: instructions in the repository's [README.md](#) (required: Node 20+, pnpm, Docker).

Good luck!